

A step-by-step guide to reinforcement learning evidence accumulation models of instrumental learning tasks

Steven Miletić, Niek Stevenson, (some other people probably), Andrew Heathcote

21 June, 2026

Introduction

In this tutorial, we will demonstrate how to design combinations of reinforcement learning and evidence accumulation models (RL-EAMs) and fit them to data from instrumental learning tasks using hierarchical Bayesian methods. We will utilize the *EMC2* package, assuming some prior knowledge of its functionalities (for reference, see the *EMC2* tutorial (Stevenson et al. 2024)).

First, let's load the required packages and data:

```
rm(list = ls())
# TMP -- install the right branch
# remotes::install_github("ampl-psych/EMC2@oo_refactor_simd", dependencies=TRUE, Ncpus=8); .rs.restartR()
# END TMP
library(EMC2)
set.seed(1) # for reproducibility

# This is some code that contains plotting functions used later on in this tutorial
if(Sys.info()[['nodename']] == 'tux22psy') {
  # load local if present
  code_url = './RL_plotting_utils.R'
} else {
  code_url <- paste0(
    "https://raw.githubusercontent.com/StevenM1/",
    "rl_eam_tutorial/refs/heads/main/RL_plotting_utils.R")
}
source(code_url)
```

From data to a basic RL-EAM: the RL-RD

While *EMC2* is flexibly designed to allow researchers to implement any mapping, in most applications of RL-EAMs, it is assumed that drift rates are a linear function of the Q-values. Key is to map the Q-values to the accumulator drift rates. This entails that we:

1. Specify the covariates in the data;
2. Apply the delta rule to each covariate separately;
3. Define the trial-wise mapping between covariates and each accumulator's drift rate.

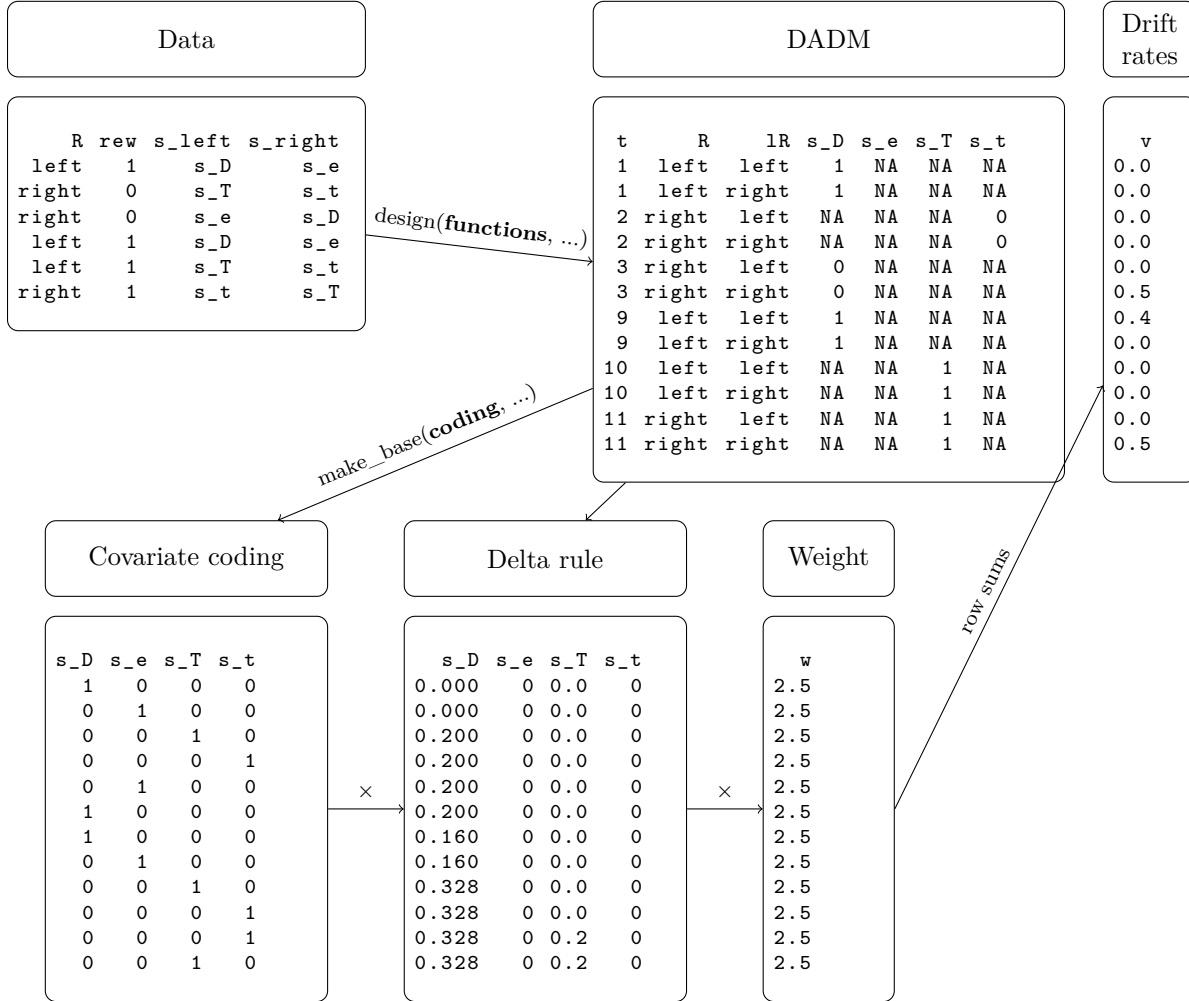


Figure 1: Overview of steps. Users need to specify functions in two places. First, a set of functions that can be applied to the **dadm** and generates the covariate columns (here, **s_D**, **s_e**, **s_T**, **s_t**). These functions are supplied to the **design()** function of **EMC2**. Second, a set of functions that can be applied to the **dadm** and generates one covariate coding. These are supplied to **make_base()**. The trial-wise drift rate is the rowsum of the updated covariate multiplied by the covariate coding and the weight.

1. Covariate specification

In this section, we start by formatting the empirical data. We start by downloading dataset 1 from (Miletić et al. 2021). It contains 24 variables; here, we aim to extract the minimum we need for modelling the data. Some are columns that *EMC2* always requires for choice tasks: columns indicating subject number (subjects), stimulus (S), and response and response time (R, rt). S and R are usually response-coded, meaning they typically refer to the response location or hand (left/right) or response button (e.g., 'z', 'm', 'left arrow key'). Let us start with extracting those:

```
load('./data/data_exp1.RData')
# exclude subjects based on criteria of Miletić et al 2021
dat <- dat[!dat$excl,]
# exclude trials without a response
dat <- dat[!is.na(dat$rt),]

# Format subjects as factor and rename column
dat$subjects <- as.factor(dat$pp)

# Format response to factor: left or right
dat$R <- factor(dat$choiceDirection, levels=c('left', 'right'))

# In order to determine which direction was 'optimal', we can check
# whether p_win_left was larger than p_win_right
dat$S <- factor(ifelse(dat$p_win_left > dat$p_win_right, 'left', 'right'),
               levels=c('left', 'right'))
head(dat[,c('subjects', 'S', 'R', 'rt')])
```

	subjects	S	R	rt
3121	7	left	left	0.750092
3122	7	left	right	0.866725
3123	7	right	right	1.100030
3124	7	left	left	0.708232
3125	7	right	right	1.174540
3126	7	left	left	1.483390

Instrumental learning tasks minimally add two pieces of information: the **feedback** (or **reward**) given on each trial, and the **symbol** that the feedback was associated with. It is not necessary that the feedback corresponds to the *chosen* symbol, as the case of experiments with counterfactual feedback (Palminteri et al. 2017; Palminteri et al. 2015) demonstrates - we only need to know to *which* symbol the feedback corresponds. Let's extract these:

```
# In this experiment, there's both an `outcome` and `reward` column;
# in this specific experiment these are the same, but in later experiments
# in the same paper these could differ. Here, we arbitrarily pick `reward`,
# and recode it to [0, 1]
dat$reward <- dat$reward / 100

# Let's extract the symbols that were presented as `s_left` and `s_right`.
# and let's add a prefix "s_"
dat$s_left <- paste0('s_', dat$appear_left)
dat$s_right <- paste0('s_', dat$appear_right)

head(dat[,c('subjects', 'S', 'R', 'rt', 'reward', 's_left', 's_right')])
```

	subjects	S	R	rt	reward	s_left	s_right
3121	7	left	left	0.750092	1	s_D	s_e
3122	7	left	right	0.866725	0	s_T	s_t
3123	7	right	right	1.100030	0	s_e	s_D
3124	7	left	left	0.708232	0	s_J	s_h
3125	7	right	right	1.174540	1	s_k	s_K
3126	7	left	left	1.483390	1	s_J	s_h

```
# based on the combination of R and s_left and s_right, we can now determine
# which symbol was chosen, and thus, corresponds to the reward.
```

Additionally, for simulating new data (but strictly speaking not for fitting), we need to be able to generate feedback in the same way that feedback was generated in the experiment. In the easiest case, feedback is deterministic and a feedback generator will simply map a chosen symbol to a corresponding reward. In more realistic examples it will require sampling a random number from a binomial or Gaussian distribution, with the parameters of that distribution varying between symbols. As such, for simulating new data and assessing quality of fit, we need to know the feedback distribution parameters for each symbol and trial.

```
dat$p_left <- dat$p_win_left
dat$p_right <- dat$p_win_right

dat <- dat[,c('subjects', 'S', 'R', 'rt', 'reward',
              's_left', 's_right', 'p_left', 'p_right')]
head(dat)
```

	subjects	S	R	rt	reward	s_left	s_right	p_left	p_right
3121	7	left	left	0.750092	1	s_D	s_e	0.70	0.30
3122	7	left	right	0.866725	0	s_T	s_t	0.80	0.20
3123	7	right	right	1.100030	0	s_e	s_D	0.30	0.70
3124	7	left	left	0.708232	0	s_J	s_h	0.65	0.35
3125	7	right	right	1.174540	1	s_k	s_K	0.40	0.60
3126	7	left	left	1.483390	1	s_J	s_h	0.65	0.35

Note that the column names of `reward`, `s_left`, `s_right`, `p_left`, and `p_right` are arbitrary - as long as the user's own functions (see the next section) refer to the correct columns, their names can be anything.

For fitting in *EMC2*, we need to create one column per symbol, with the values indicating the feedback. If no feedback was given on the symbol, the value should be `NA`. An intuitive option would be to add such columns to the data. However, all columns in the data (except for `rt` and `R`) are assumed to be static factors of the design. Keep in mind that we also want to be able to *simulate* new data. And once we simulate new data, the covariate columns should depend on the simulated behavior - that is, the covariate columns are *not* factors of the experimental design, and should not be treated as such. For this reason, we should not add the covariate columns to the data directly, but rather define *functions* that create covariate columns based on the data. When simulating new data, these functions are re-applied after each trial, ensuring that the covariate columns are correct for simulated data. Here's the idea:

```
make_covariate_column <- function(data, covariate_symbol) {
  # Which response symbol (RS) was chosen?
  RS <- ifelse(data$R=='left', data$s_left, data$s_right)
  data[,covariate_symbol] <- NA
  data[RS == covariate_symbol, covariate_symbol] <- data[RS == covariate_symbol, 'reward']
  data[[covariate_symbol]]
}

tmp <- dat
```

```
tmp$s_D <- make_covariate_column(tmp, "s_D")
tmp$s_e <- make_covariate_column(tmp, "s_e")
tmp$s_t <- make_covariate_column(tmp, "s_t")
head(tmp)
```

	subjects	S	R	rt	reward	s_left	s_right	p_left	p_right	s_D	s_e
3121	7	left	left	0.750092	1	s_D	s_e	0.70	0.30	1	NA
3122	7	left	right	0.866725	0	s_T	s_t	0.80	0.20	NA	NA
3123	7	right	right	1.100030	0	s_e	s_D	0.30	0.70	0	NA
3124	7	left	left	0.708232	0	s_J	s_h	0.65	0.35	NA	NA
3125	7	right	right	1.174540	1	s_k	s_K	0.40	0.60	NA	NA
3126	7	left	left	1.483390	1	s_J	s_h	0.65	0.35	NA	NA

	s_t
3121	NA
3122	0
3123	NA
3124	NA
3125	NA
3126	NA

```
# To facilitate, we can wrap this with a function generator -
# that is, a function that returns a function, one for each column
# Each of those returned function generates a covariate column
# with the appropriate structure.
covariate_column_generator <- function(col_name) {
  function(data) {
    # Which response symbol was chosen?
    RS <- ifelse(data$R=='left', data$s_left, data$s_right)
    data[,col_name] <- NA
    data[RS == col_name, col_name] <- data[RS == col_name, 'reward']
    data[[col_name]]
  }
}

all_symbols <- unique(c(dat$s_left, dat$s_right))
make_covariate_columns <- setNames(lapply(all_symbols,
                                          covariate_column_generator), all_symbols)
```

Now, `make_covariate_columns` is a list of functions that should be applied to the data. We can tell *EMC2* to do this by providing the functions to the `design`:

```
design_RDM <- design(model=RDM,
                    data=dat,
                    functions=make_covariate_columns,
                    matchfun=function(d) d$s==d$lR,
                    formula=list(B ~ 1, v ~ 1, t0 ~ 1),
                    report_p_vector = FALSE)

# Once the design is combined with the data, the functions are applied.
# and we can find all covariate columns in the `dadm`:
emc <- make_emc(dat, design_RDM)
head(emc[[1]]$data[[1]])
```

subjects	S	R	rt	reward	s_left	s_right	p_left	p_right	trials
----------	---	---	----	--------	--------	---------	--------	---------	--------

1	7	left	left	0.7500000	1	s_D	s_e	0.7	0.3	1
2	7	left	left	0.7500000	1	s_D	s_e	0.7	0.3	1
3	7	left	right	0.8666667	0	s_T	s_t	0.8	0.2	2
4	7	left	right	0.8666667	0	s_T	s_t	0.8	0.2	2
5	7	right	right	1.1000000	0	s_e	s_D	0.3	0.7	3
6	7	right	right	1.1000000	0	s_e	s_D	0.3	0.7	3

	lR	lM	winner	s_D	s_T	s_e	s_J	s_k	s_h	s_t	s_K
1	left	TRUE	TRUE	1	NA	NA	NA	NA	NA	NA	NA
2	right	FALSE	FALSE	1	NA	NA	NA	NA	NA	NA	NA
3	left	TRUE	FALSE	NA	NA	NA	NA	NA	NA	0	NA
4	right	FALSE	TRUE	NA	NA	NA	NA	NA	NA	0	NA
5	left	FALSE	FALSE	0	NA	NA	NA	NA	NA	NA	NA
6	right	TRUE	TRUE	0	NA	NA	NA	NA	NA	NA	NA

2. Apply the delta rule

Handling trialwise covariates works through `trend` objects in *EMC2*. In this object, you specify (1) the covariate of interest, (2) a kernel to apply to the covariate, and (3) which decision parameter is informed by the resulting covariate, and (4) the functional form of the mapping between the resulting covariate and the decision parameters, referred to as the ‘base’. Now that we have prepared the data-augmented design matrix (`dadm`) (so that it contains all covariates as columns), we can now specify a `trend` object. The kernel that we want to apply is the `delta` rule, and it should inform drift rates `v` linearly, so we use a linear `base`.

```
delta_kernel <- make_kernel(cov_names=all_symbols, type='delta')
lin_base <- make_base(target_parameter = 'v', type='lin', kernel=delta_kernel,
                      phase='posttransform')
trend <- make_trend(lin_base)

design_RDM <- design(model=RDM,
                    data=dat,
                    functions=make_covariate_columns,
                    matchfun=function(d) d$S==d$lR,
                    formula=list(B ~ 1, v ~ 1, t0 ~ 1),
                    trend=trend,
                    report_p_vector = FALSE)

emc <- make_emc(dat, design_RDM)

# To illustrate the effect
kernel_pars <- c('v.q0'=0, 'v.alpha'=0.2)
head(cbind(emc[[1]]$data[[1]][,all_symbols], apply_kernel(kernel_pars, emc = emc, subject=1)))
```

	s_D	s_T	s_e	s_J	s_k	s_h	s_t	s_K		1	2	3	4	5	6	7	8
1	1	NA	NA	NA	NA	NA	NA	NA	0.0000000	0	0	0	0	0	0	0	0
2	1	NA	NA	NA	NA	NA	NA	NA	0.0000000	0	0	0	0	0	0	0	0
3	NA	NA	NA	NA	NA	NA	0	NA	0.5792597	0	0	0	0	0	0	0	0
4	NA	NA	NA	NA	NA	NA	0	NA	0.5792597	0	0	0	0	0	0	0	0
5	0	NA	NA	NA	NA	NA	NA	NA	0.5792597	0	0	0	0	0	0	0	0
6	0	NA	NA	NA	NA	NA	NA	NA	0.5792597	0	0	0	0	0	0	0	0

3. Define the trial-wise mapping between covariates and each accumulator

By default, *EMC2* applies the base to all updated covariates (i.e., it multiplies the base with the rowsums of the updated kernel). However, on each trial, only two stimuli were shown, so only they should influence the respective drift rates. Therefore, we need to create a mapping between covariates and accumulators. Such a coding is a matrix specifying the weight of each covariate for each drift rate - most will be 0. The simplest type of coding corresponds to an RL-RD, which just maps 1 covariate to 1 accumulator in each row:

```
make_RL_RD_coding <- function(dadm, cov_names) {
  dadm$IS <- NA
  dadm[dadm$LR=='left', 'IS'] <- dadm[dadm$LR=='left', 's_left']
  dadm[dadm$LR=='right', 'IS'] <- dadm[dadm$LR=='right', 's_right']

  # Return 1 if the column name matches the accumulator's latent stimulus
  # and 0 otherwise
  coding <- sapply(cov_names, function(col) ifelse(dadm$IS == col, 1, 0))
  coding
}

head(make_RL_RD_coding(emc[[1]]$data[[1]], all_symbols))
```

```
      s_D s_T s_e s_J s_k s_h s_t s_K
[1,]  1  0  0  0  0  0  0  0
[2,]  0  0  1  0  0  0  0  0
[3,]  0  1  0  0  0  0  0  0
[4,]  0  0  0  0  0  0  1  0
[5,]  0  0  1  0  0  0  0  0
[6,]  1  0  0  0  0  0  0  0
```

```
# Combine with trend and design
delta_kernel <- make_kernel(cov_names=all_symbols, type='delta')
lin_base <- make_base(target_parameter = 'v', type='lin', kernel=delta_kernel,
                      phase='posttransform', coding=list(w=make_RL_RD_coding))
trend <- make_trend(lin_base)

# trend <- make_trend(par_names='v',
#                      cov_names=list(all_symbols),
#                      kernels='delta',
#                      bases='lin',
#                      phase='posttransform',
#                      coding = list(make_RL_RD_coding))

design_RDM <- design(model=RDM,
                    data=dat,
                    functions=make_covariate_columns,
                    matchfun=function(d) d$S==d$LR,
                    formula=list(B ~ 1, v ~ 1, t0 ~ 1),
                    trend=trend,
                    constants=c('v.q0'=0),
                    report_p_vector = FALSE)

emc <- make_emc(dat, design_RDM)
```

Note how the drift rate is now only influenced by the covariate that corresponds to the symbol that the accumulator codes for.

We are now ready to fit the RL-RD!

```
# emc <- fit(emc, cores_per_chain=20, fileName='./samples/exp1_subset_rlrld.RData')
```

```
# load('./samples/exp1_subset_rlrld.RData')
# ppC <- predict(emc, n_post=20, n_cores=50)
# plot_exp1(dat=dat, pp=ppC)
```

4. Feedback generator

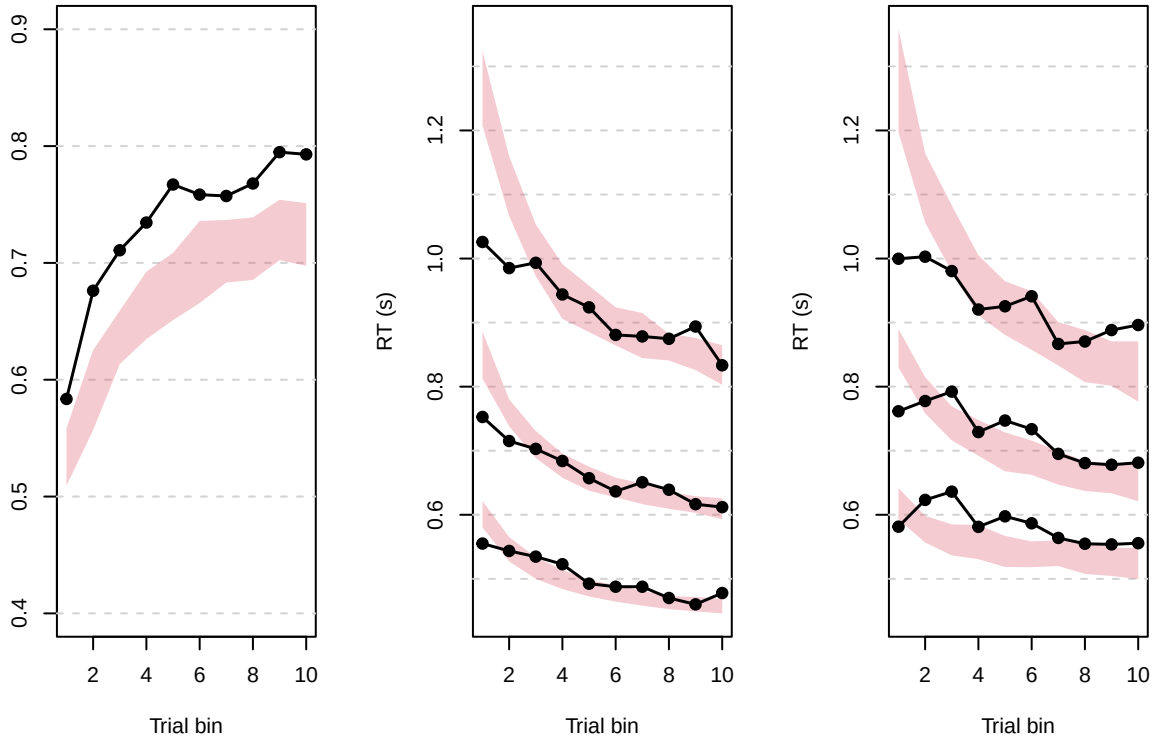
ToDo: Add a feedback generator function. This requires more work on EMC2 for a principled implementation. For now, let's try this

```
feedback_generator <- function(d) {
  data <- d[d$L1R==levels(d$L1R)[1],]
  p_chosen <- ifelse(data$R=='left', data$p_left, data$p_right)
  rewards <- rbinom(nrow(data), 1, p_chosen)
  rep(rewards, each=nlevels(d$L1R))
  # alternatively, map symbol to reward probability or reward function here.
  # can change trialwise/subjectwise etc
}
trend$kernel$feedback <- list('reward'=feedback_generator)

design_RDM <- design(model=RDM,
  data=dat,
  functions=make_covariate_columns,
  matchfun=function(d) d$S==d$L1R,
  formula=list(B ~ 1, v ~ 1, t0 ~ 1),
  trend=trend,
  constants=c('v.q0'=0),
  report_p_vector = FALSE)

emc <- make_emc(dat, design_RDM)
emc <- fit(emc, cores_per_chain=5, fileName='./samples/exp1_rlrld.RData')
```

```
load('./samples/exp1_rlrld.RData')
ppU <- predict(emc, n_cores=25, conditional_on_data=FALSE, return_trialwise_parameters=TRUE)
plot_exp1(dat=dat, pp=ppU[is.finite(ppU$rt),])
```

Example: RL-DDM

The above illustrates the basics, and with these, it is straightforward to construct other RL-EAMs. In most cases, it is a matter of changing the coding function, and potentially the EAM itself. For example, a popular choice is the RL-DDM, which is the following:

```
make_RL_DDM_coding <- function(dadm, cov_names) {
  # Here, we simply find the symbol on the left and right,
  # and subtract one from the other
  coding_left <- sapply(cov_names, function(col) ifelse(dadm$s_left == col, 1, 0))
  coding_right <- sapply(cov_names, function(col) ifelse(dadm$s_right == col, 1, 0))
  coding <- coding_right - coding_left
  coding
}

feedback_generator_ddm <- function(d) {
  p_chosen <- ifelse(d$R == 'left', d$p_left, d$p_right)
  rewards <- rbinom(nrow(d), 1, p_chosen)
  if('lR' %in% names(d)) {
    rewards <- rep(rewards, each=nlevels(d$lR))
  }
  rewards
}

# Combine with trend and design
```

```

delta_kernel_DDM <- make_kernel(cov_names=all_symbols, type='delta', at=NULL)
lin_base_DDM <- make_base(target_parameter = 'v', type='lin', kernel=delta_kernel,
                          phase='posttransform', coding=list(w=make_RL_DDM_coding))
trend_DDM <- make_trend(lin_base_DDM)
trend_DDM$kernel$kernel$feedback <- list('reward'=feedback_generator_ddm)

design_DDM <- design(model=DDM,
                    data=dat,
                    functions=make_covariate_columns,
                    matchfun=function(d) d$S==d$I,
                    formula=list(a ~ 1, v ~ 1, t0 ~ 1),
                    trend=trend_DDM,
                    constants=c('v.q0'=0),
                    report_p_vector = FALSE)

emc <- make_emc(dat, design_DDM)
head(attr(emc[[1]]$data[[1]], 'covariate_coding')[[1]])

```

```

      s_D s_T s_e s_J s_k s_h s_t s_K
[1,]  -1  0  1  0  0  0  0  0
[2,]   0 -1  0  0  0  0  1  0
[3,]   1  0 -1  0  0  0  0  0
[4,]   0  0  0 -1  0  1  0  0
[5,]   0  0  0  0 -1  0  0  1
[6,]   0  0  0 -1  0  1  0  0

```

```

emc <- fit(emc, cores_per_chain=5, fileName='./samples/exp1_rlDDM.RData')

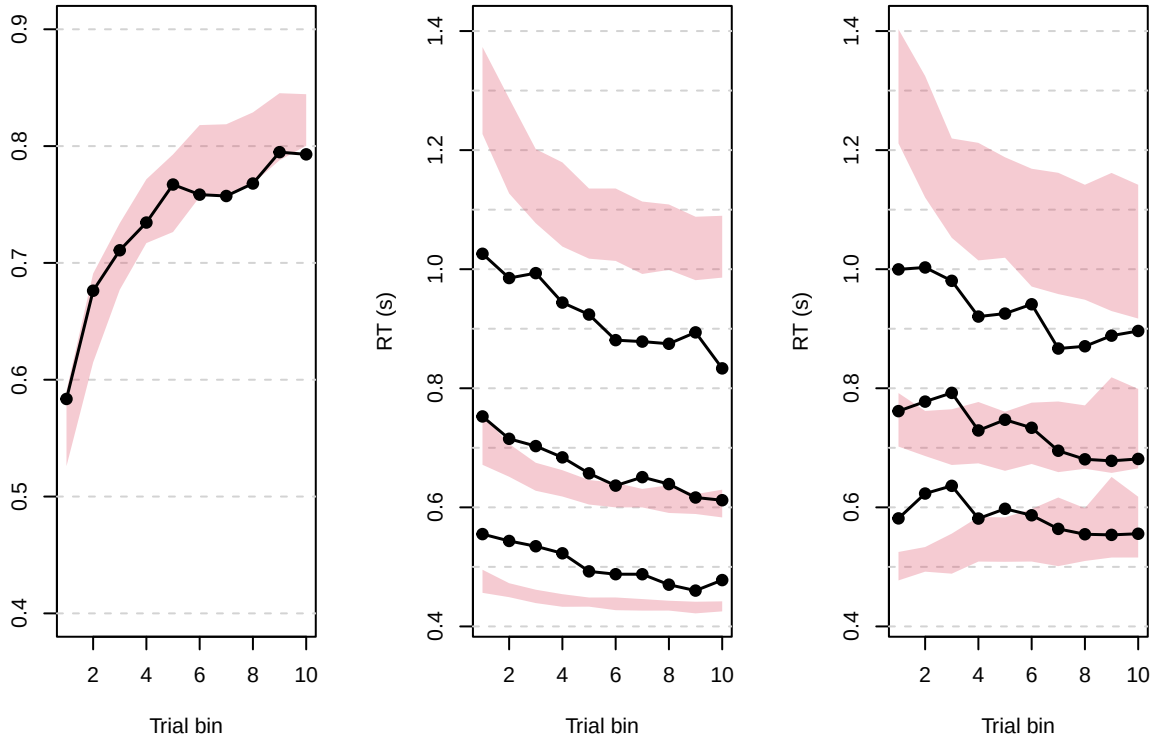
```

However, the RL-DDM does not fit the response time distributions very well:

```

## plot here
load('./samples/exp1_rlDDM.RData')
ppU <- predict(emc, n_cores=50)
plot_exp1(dat=dat, pp=ppU[is.finite(ppU$rt),])

```



Example: RL-ARD

An alternative option is the RL-ARD, which combines a difference and a sum coding in a race architecture:

```
make_RL_ARD_difference <- function(dadm, cov_names) {
  dadm$lS <- NA
  dadm[dadm$lR=='left', 'lS'] <- dadm[dadm$lR=='left', 's_left']
  dadm[dadm$lR=='right', 'lS'] <- dadm[dadm$lR=='right', 's_right']

  dadm$lSother <- NA
  dadm[dadm$lR=='left', 'lSother'] <- dadm[dadm$lR=='left', 's_right']
  dadm[dadm$lR=='right', 'lSother'] <- dadm[dadm$lR=='right', 's_left']

  # Return 1 if the column name matches the accumulator's latent stimulus
  # Return -1 if the column name matches the *other* accumulator's latent stimulus
  # and 0 otherwise
  codinglS <- sapply(cov_names, function(col) ifelse(dadm$lS == col, 1, 0))
  codinglSother <- sapply(cov_names, function(col) ifelse(dadm$lSother == col, 1, 0))
  codinglS-codinglSother
}

make_RL_ARD_sum <- function(dadm, cov_names) {
  dadm$lS <- NA
  dadm[dadm$lR=='left', 'lS'] <- dadm[dadm$lR=='left', 's_left']
  dadm[dadm$lR=='right', 'lS'] <- dadm[dadm$lR=='right', 's_right']
}
```

```

dadm$lSother <- NA
dadm[dadm$lR=='left', 'lSother'] <- dadm[dadm$lR=='left', 's_right']
dadm[dadm$lR=='right', 'lSother'] <- dadm[dadm$lR=='right', 's_left']

# Return 1 if the column name matches the accumulator's latent stimulus
# Return 1 if the column name matches the *other* accumulator's latent stimulus
# and 0 otherwise
codinglS <- sapply(cov_names, function(col) ifelse(dadm$lS == col, 1, 0))
codinglSother <- sapply(cov_names, function(col) ifelse(dadm$lSother == col, 1, 0))
codinglS+codinglSother
}

# Combine with trend and design
base1 <- make_base('v', 'lin', kernel=delta_kernel,
                  coding=list('d'=make_RL_ARD_difference), phase='posttransform')
base2 <- make_base('v', 'lin', kernel=delta_kernel,
                  coding=list('s'=make_RL_ARD_sum), phase='posttransform')
trend <- make_trend(base1, base2)
trend$kernel$feedback <- list('reward'=feedback_generator)

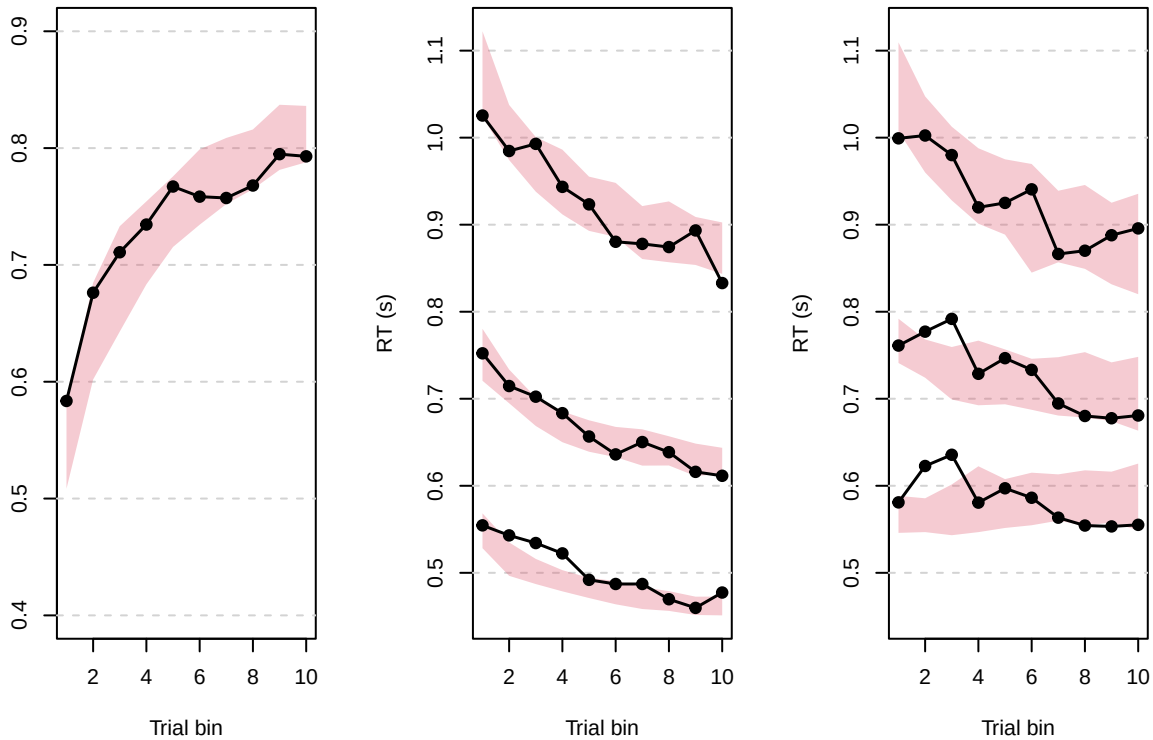
design_RDM <- design(model=RDM,
                   data=dat,
                   functions=make_covariate_columns,
                   matchfun=function(d) d$lS==d$lR,
                   formula=list(B ~ 1, v ~ 1, t0 ~ 1),
                   trend=trend,
                   constants=c('v.q0'=0),
                   report_p_vector = FALSE)

emc <- make_emc(dat, design_RDM)

emc <- fit(emc, cores_per_chain=5, fileName='./samples/exp1_rlard.RData')

load('./samples/exp1_rlard.RData')
ppU <- predict(emc, n_cores=15, return_trialwise_parameters=TRUE)
plot_exp1(dat=get_data(emc), pp=ppU[is.finite(ppU$rt),])

```



Example 1: Factors on parameters. The speed-accuracy trade-off

Many research questions ask which cognitive processes are affected by a manipulation. A classic example in the decision-making literature are manipulations intended to affect how people adjust to varying demands of speed and accuracy. This section reproduces the results from experiment 2 of (?). In this experiment, participants performed an instrumental learning task. Prior to each trial, they were instructed to emphasise either response speed or response accuracy. The analyses then fit an RL-ARD to test whether this cue-based manipulation affected response thresholds and/or response urgency. Here, we first fit a model where both parameters vary with cue:

```
# Load and format the data
load('./data/data_exp3.RData')
dat <- dat[!dat$excl,]
dat <- dat[!is.na(dat$rt),]
dat$subjects <- as.factor(dat$pp)
dat$R <- factor(dat$choice_direction, levels=c(0, 1), labels=c('left', 'right'))
dat$S <- factor(ifelse(dat$p_win_left>dat$p_win_right, 'left', 'right'),
               levels=c('left', 'right'))
dat$reward <- dat$outcome
dat$s_left <- paste0('s_', dat$stimulus_symbol_left)
dat$s_right <- paste0('s_', dat$stimulus_symbol_right)
dat$p_left <- dat$p_win_left
dat$p_right <- dat$p_win_right
dat$cue <- as.factor(dat$cue)
```

```

# keep track of how often a stimulus pair has been shown, within each block & participant
dat$pair <- paste(pmin(dat$s_left, dat$s_right),
                 pmax(dat$s_left, dat$s_right),
                 sep = "_")
dat$pair_count <- as.numeric(ave(dat$pair, dat$subjects, dat$block_nr, dat$pair,
                                FUN = seq_along))
dat$trial_bin <- as.numeric(cut(dat$pair_count, 10))
dat <- dat[,c('subjects', 'S', 'R', 'rt', 'reward', 's_left', 's_right', 'p_left', 'p_right', 'cue', 'trial_bin')]
head(dat)

```

	subjects	S	R	rt	reward	s_left	s_right	p_left	p_right	cue
6	1	left	right	0.4918751	0	s_C	s_c	0.8	0.2	SPD
19	1	left	left	0.7085768	1	s_D	s_g	0.7	0.3	ACC
32	1	left	right	0.4918683	0	s_f	s_x	0.6	0.4	SPD
45	1	left	left	0.6012016	1	s_f	s_x	0.6	0.4	ACC
58	1	right	right	0.6845398	1	s_g	s_D	0.3	0.7	ACC
71	1	left	left	0.4585508	1	s_f	s_x	0.6	0.4	SPD

	trial_bin
6	1
19	1
32	1
45	1
58	1
71	1

```

all_symbols <- unique(c(dat$s_left, dat$s_right))
make_covariate_columns <- setNames(lapply(all_symbols,
                                           covariate_column_generator), all_symbols)

kernel <- make_kernel(all_symbols, 'delta')
base1 <- make_base('v', 'lin', kernel=kernel, coding=list('d'=make_RL_ARD_difference),
                  phase='posttransform')
base2 <- make_base('v', 'lin', kernel=kernel, coding=list('d'=make_RL_ARD_sum),
                  phase='posttransform')
trend <- make_trend(base1, base2)
trend$kernel$feedback <- list('reward'=feedback_generator)

ADmat <- matrix(c(1, -1), ncol=1, dimnames=list(NULL, c('a-s')))
design_RDM <- design(model=RDM,
                    data=dat,
                    functions=make_covariate_columns,
                    matchfun=function(d) d$S==d$R,
                    contrasts=list(cue=ADmat),
                    formula=list(B ~ cue, v ~ cue, t0 ~ 1),
                    trend=trend,
                    constants=c('v.q0'=0),
                    report_p_vector=TRUE)

```

Sampled Parameters:

[1] "B"	"B_cuea-s"	"v"	"v_cuea-s"
[5] "t0"	"v.alpha"	"v.w_d_delta"	"v.w_d_delta_2"

Design Matrices:

```
$B
cue B B_cuea-s
ACC 1      1
SPD 1     -1
```

```
$v
cue v v_cuea-s
ACC 1      1
SPD 1     -1
```

```
$t0
t0
1
```

```
$v.q0
v.q0
1
```

```
$v.alpha
v.alpha
1
```

```
$v.w_d_delta
v.w_d_delta
1
```

```
$v.w_d_delta_2
v.w_d_delta_2
1
```

```
$A
A
1
```

```
$s
s
1
```

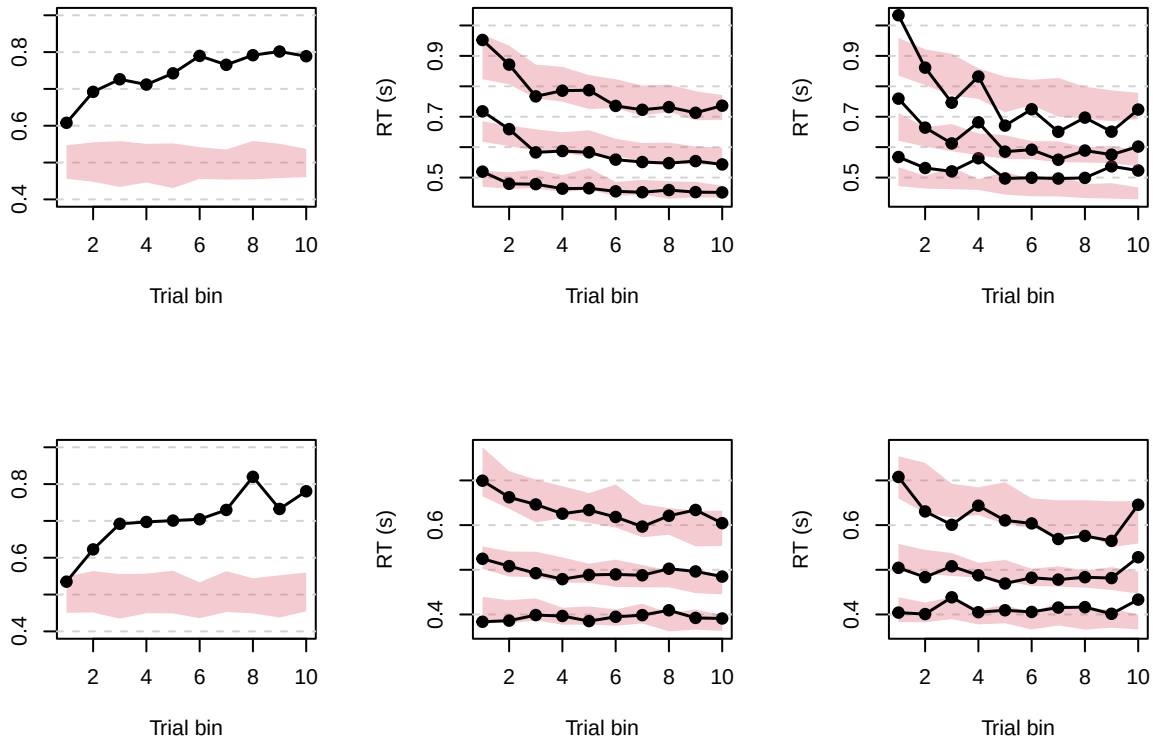
```
emc <- make_emc(dat, design_RDM)
```

```
emc <- fit(emc, cores_per_chain=5, fileName='./samples/exp3_sat_rlard.RData')
```

To do: plotting of results, split by SAT

```
load('./samples/exp3_sat_rlard.RData')
pp <- predict(emc, n_cores=15, return_trialwise_parameters=TRUE)

dat <- get_data(emc)
par(mfrow=c(2,3))
plot_exp3(dat=dat[dat$cue=='ACC',], pp=pp[pp$cue=='ACC'&is.finite(pp$rt),], do.par=FALSE)
plot_exp3(dat=dat[dat$cue=='SPD',], pp=pp[pp$cue=='SPD'&is.finite(pp$rt),], do.par=FALSE)
```



We can compare the model above with a model that allows only the threshold to vary with cues:

```
design_RDM2 <- design(model=RDM,
  data=dat,
  functions=make_covariate_columns,
  matchfun=function(d) d$S==d$I,
  contrasts=list(cue=ADmat),
  formula=list(B ~ cue, v ~ 1, t0 ~ 1),
  trend=trend,
  constants=c('v.q0'=0),
  report_p_vector = TRUE)
```

Sampled Parameters:

```
[1] "B"          "B_cuea-s"    "v"           "t0"
[5] "v.alpha"    "v.w_d_delta" "v.w_d_delta_2"
```

Design Matrices:

```
$B
  cue B B_cuea-s
ACC 1      1
SPD 1     -1
```

```
$v
v
1
```



```

$t0
  t0
  1

$v.q0
  v.q0
  1

$v.alpha
  v.alpha
  1

$v.w_d_delta
  v.w_d_delta
  1

$v.w_d_delta_2
  v.w_d_delta_2
  1

$A
  A
  1

$s
  s
  1

```

```

emc <- make_emc(dat, design_RDM)
emc <- fit(emc, cores_per_chain=5, fileName='./samples/exp3_sat_rlard_model2.RData')

## plot by SAT condition, get BayesFactors

```

```

emc1 <- EMC2::loadRData('./samples/exp3_sat_rlard.RData')
emc2 <- EMC2::loadRData('./samples/exp3_sat_rlard_model2.RData')

compare(sList=list(emc1=emc1, emc2=emc2))

```

	MD	wMD	DIC	wDIC	BPIC	wBPIC	EffectiveN	meanD	Dmean	minD
emc1	1375	0.045	1097	0.026	1213	0.01	116	981	911	865
emc2	1369	0.955	1090	0.974	1204	0.99	114	976	906	862

```

# which demonstrates that the threshold-only model wins

```

Example 2: Positive and negative learning rates

The standard delta rule assumes people adjust Q-values equally when reward is higher than expected compared to when it is lower than expected. However, several research papers suggest that people show an optimism bias: Positive prediction errors lead to larger Q-value adjustments than negative adjustments. This is formalized by allowing for two separate learning rates:

$$PE_t = r_t - Q_t$$

$$Q_{t+1} = Q_t + \begin{cases} \alpha^+ \cdot PE_t & \text{if } PE_t > 0 \\ \alpha^- \cdot PE_t & \text{if } PE_t < 0 \end{cases}$$

One example paper is (?), who openly share the data, which we can use to demonstrate:

```
load('./data/lefebvre2017/lefebvre_exp1.RData')

all_symbols <- unique(c(dat$s_left, dat$s_right))
make_covariate_columns <- setNames(lapply(all_symbols,
                                          covariate_column_generator), all_symbols)

kernel <- make_kernel(all_symbols, 'delta')
base1 <- make_base('v', 'lin', kernel=kernel, phase='posttransform', coding=list('d'=make_RL_ARD_difference))
base2 <- make_base('v', 'lin', kernel=kernel, phase='posttransform', coding=list('s'=make_RL_ARD_sum))
trend <- make_trend(base1, base2)
trend$kernel$feedback <- list('reward'=feedback_generator)

ADmat <- matrix(c(1, -1), ncol=1, dimnames=list(NULL, c('a-s')))
design_RDM <- design(model=RDM,
                    data=dat,
                    functions=make_covariate_columns,
                    contrasts=list(cue=ADmat),
                    formula=list(B ~ 1, v ~ 1, t0 ~ 1),
                    trend=trend,
                    constants=c('v.q0'=0),
                    report_p_vector=TRUE)
```

Sampled Parameters:

```
[1] "B"      "v"      "t0"     "v.alpha" "v.w_d"  "v.w_s"
```

Design Matrices:

```
$B
```

```
B
```

```
1
```

```
$v
```

```
v
```

```
1
```

```
$t0
```

```
t0
```

```
1
```

```
$v.q0
```

```
v.q0
```

```
1
```

```
$v.alpha
```

```
v.alpha
```

```
1
```

```
$v.w_d
```

```
v.w_d  
1
```

```
$v.w_s  
v.w_s  
1
```

```
$A  
A  
1
```

```
$s  
s  
1
```

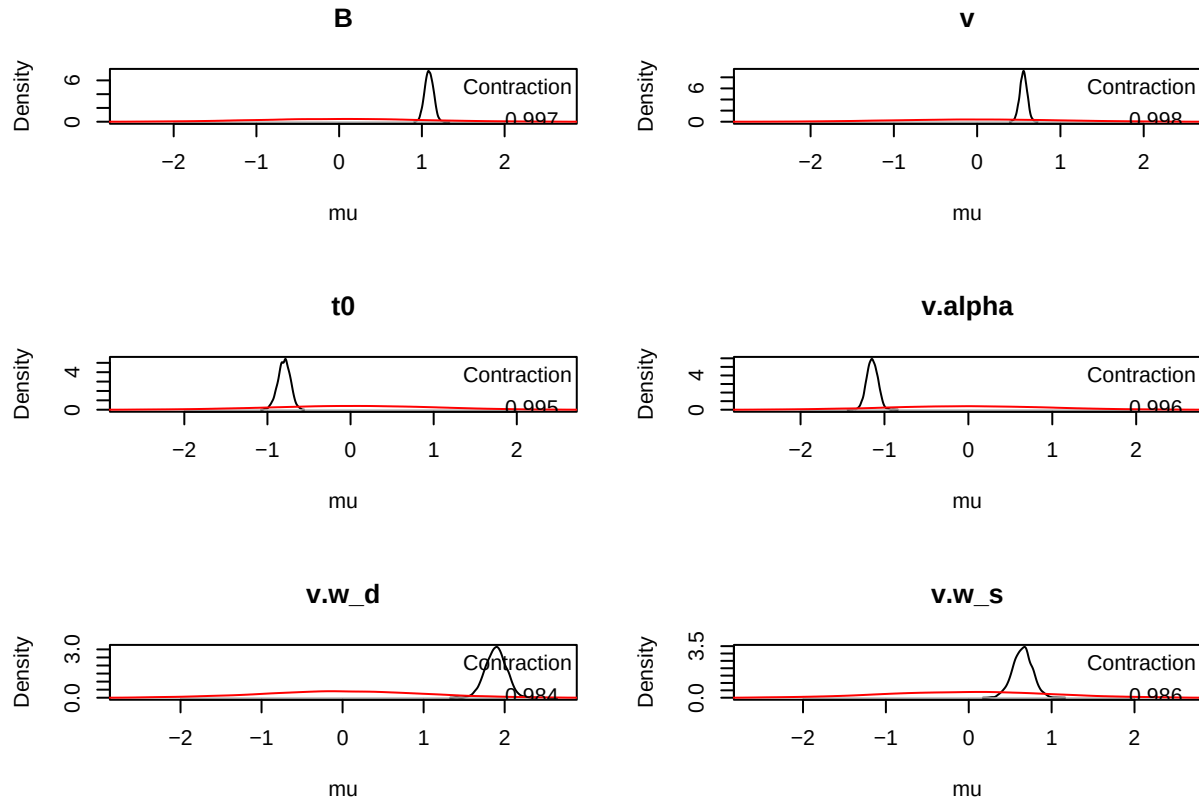
```
emc <- make_emc(dat, design_RDM)  
emc <- fit(emc, cores_per_chain=5, fileName='./samples/lefebvre2017exp1_2lr.RData')
```

We can inspect the group-level posterior parameters, which indeed suggest that there is a difference between the positive (group-level mean ~ 0.17) and negative (~ -0.064) learning rates:

```
credint(emc)
```

```
$mu  
      2.5%   50%  97.5%  
B      0.987  1.086  1.186  
v      0.467  0.556  0.641  
t0     -0.949 -0.797 -0.664  
v.alpha -1.275 -1.147 -1.023  
v.w_d   1.641  1.894  2.133  
v.w_s   0.411  0.650  0.883
```

```
plot_pars(emc)
```



While this way of parameterising the model can be used for model comparisons (to demonstrate that two learning rates are needed for a parsimonious account of the data), it does not directly estimate the *difference* in learning rates - which is likely the target of inference. For that purpose, a better parameterisation would be to fit the mean and difference of the learning rates, such that $\alpha_{pos} = \alpha_{mean} + 0.5 * \alpha_{diff}$, and $\alpha_{neg} = \alpha_{mean} - 0.5 * \alpha_{diff}$. This way, one can specify a prior on the difference, and obtain participant-wise estimates of it. For factors that vary between conditions (and thus, between rows), EMC2 uses R's formula language. However, these two learning rates don't vary between rows of the design. Instead, this is a linear reparameterisation. In EMC2, we can do this as follows:

```
# design_RDM <- design(model=RDM,
#                       data=dat,
#                       functions=make_covariate_columns,
#                       contrasts=list(cue=ADmat),
#                       formula=list(B ~ 1, v ~ 1, t0 ~ 1),
#                       trend=trend,
#                       constants=c('v.q0'=0),
#                       report_p_vector=TRUE,
#                       reparameterisations = list('v.alphaPos'=c('v.alphaMean'=1, 'v.alphaDiff'=0.5),
#                                                  'v.alphaNeg'=c('v.alphaMean'=1, 'v.alphaDiff'=-0.5)),
#                       #
# emc <- make_emc(dat, design_RDM)
# emc <- fit(emc, cores_per_chain=20, fileName='./samples/lefebvre2017exp1_2lr-reparam.RData')
#
# # now, we can visualize the posterior over the learning-rate difference:
# credint(emc)
# plot_pars(emc)
```

```
#  
# # and extract subject-wise learning rate differences:  
# head(parameters(emc, selection='alpha'))
```

Example 3: Between-participant differences

The (?) paper addresses a between-subject differences question.

```
# group design
```

Example 4: Counterfactual feedback

- Miletić, Steven, Russell J. Boag, Anne C. Trutti, Niek Stevenson, Birte U. Forstmann, and Andrew Heathcote. 2021. “A New Model of Decision Processing in Instrumental Learning Tasks.” *eLife* 10 (January): 1–33. <https://doi.org/10.7554/eLife.63055>.
- Palminteri, Stefano, Mehdi Khamassi, Mateus Joffily, and Giorgio Coricelli. 2015. “Contextual Modulation of Value Signals in Reward and Punishment Learning.” *Nature Communications* 6. <https://doi.org/10.1038/ncomms9096>.
- Palminteri, Stefano, Germain Lefebvre, Emma J. Kilford, and Sarah-Jayne Blakemore. 2017. “Confirmation Bias in Human Reinforcement Learning: Evidence from Counterfactual Feedback Processing.” *PLOS Computational Biology* 13 (8): e1005684. <https://doi.org/10.1371/journal.pcbi.1005684>.
- Stevenson, Niek, Michelle Donzallaz, Reilly James Innes, Birte Forstmann, Dora Matzke, and Andrew Heathcote. 2024. “EMC2: An R Package for Cognitive Models of Choice.” <https://doi.org/10.31234/osf.io/2e4dq>.